

Project Overview: Speech Emotion Recognition for Astronaut–AI Communication

1. Technology Stack

Details:

We use the following stack:

- Programming Language: Python
- Frameworks: PyTorch, Hugging Face Transformers, Torchaudio
- Data Handling: Pandas, scikit-learn
- Training Orchestration: Hugging Face Trainer API
- Visualization & Evaluation: Matplotlib, scikit-learn metrics
- Deployment/Runtime: ROS2 for real-time communication

Why: This stack is chosen for its maturity, ecosystem support, and ability to efficiently handle audio-based machine learning tasks.

2. Software Used

Details:

- PyTorch & Hugging Face Transformers → training & inference
- scikit-learn → metrics (accuracy, F1)
- Matplotlib → confusion matrix visualization
- ROS2 (rclpy) → runtime integration for robotic systems
- Docker (optional) → environment reproducibility

Why: These software tools ensure reproducibility, robust model training, and seamless integration with robotic/AI communication pipelines.

3. Hardware Used

Details:

- GPU (4–8 GB NVIDIA GPU) → training fine-tuned models
- CPU → inference on embedded/robotics hardware
- Microphones + audio interface → capturing astronaut voice

Why: Hardware is selected for training efficiency and deployment feasibility in constrained environments such as spacecraft systems.

4. Model Used

Details:

We use Wav2Vec2ForSequenceClassification (from Hugging Face).

Why: This model is pretrained on large-scale speech data and requires only fine-tuning, making it efficient and accurate for emotion recognition.

5. Data Used

Details:

We train primarily on the RAVDESS dataset, with placeholders for expansion to IEMOCAP.

Why: RAVDESS provides labeled emotional speech across key categories, offering a balanced and widely used benchmark for speech emotion recognition.

6. Training Method

Details:

- Supervised fine-tuning of pretrained Wav2Vec2
- Data augmentations: resampling, bandpass limiting, noise injection, packet loss simulation
- Freezing encoder layers while fine-tuning classification head and last N layers
- Gradient checkpointing + mixed precision (fp16) training
- Stratified dataset splits (train/validation/test)

Why: This method ensures robustness to real-world astronaut communication challenges, reduces compute requirements, and improves model generalization.